Python Programming & AI

Level 1

Task 1: CLI Expense Tracker with Persistent Storage

Modules 1, 2 & 4

Description:

- Build a command-line expense tracker that accepts user input for amount, category, and date.
- Use lists and dictionaries to store entries in memory during a session.
- Persist entries to a local JSON file so data survives between runs, and reload it on startup.

Tools & Libraries:

Python | json

Covered Topics

Variables & Data Types | Lists & Dictionaries | File I/O | Basic CLI Design

Bonus:

- Add a monthly summary report using f-strings and basic aggregation (sum, average per category).

Task 2: Student Grade Processor with Custom Functions

Module 3

Description:

- Write a set of modular functions that calculate average, highest/lowest score, and pass/fail status for a list of student records.
- Organize the code into a separate importable module rather than a single file.
- Add input validation for invalid scores (e.g., negative numbers or scores above 100).

Tools & Libraries:

Python

Covered Topics

Functions & Parameters | Modules & Imports | PEP 8 Style | Input Validation

Bonus:

- Raise and handle a custom exception class for invalid score input, and add type hints to your function signatures.

Task 3: Automated File Organizer

Module 4

Description:

- Write a script that scans a target directory and automatically sorts files into subfolders based on file extension.
- Never overwrite an existing file at the destination — if a filename conflict occurs, rename the incoming file (e.g., append a number or timestamp) rather than replacing what's already there. This is a core requirement, not optional.
- Print a clear summary of every move operation (filename, source, destination) to the console.
- Handle edge cases gracefully: duplicate filenames, missing permissions, empty folders.

Tools & Libraries:

Python | os | shutil | pathlib

Covered Topics

File Handling | Exception Handling | Automation | Safe File Operations

Bonus:

Replace console output with proper logging to a file, and add a dry-run mode that previews changes without moving files. Reading ahead: schedule the script using cron or Task Scheduler.

Level 2

Task 4: Object-Oriented Inventory Management System

Module 5

Description:

- Design a class-based inventory system with a base Product class and at least one subclass (e.g., PerishableProduct) that adds genuinely distinct behavior (e.g., expiry tracking), not just an empty extension of the base class.
- Use Python's @property decorator where validation or controlled access is actually needed (e.g., preventing negative stock values).
- Support core operations: add stock, remove stock, check low-inventory alerts, generate a formatted report.

Tools & Libraries:

Python | OOP (classes, inheritance, properties)

Covered Topics

Object-Oriented Programming | Inheritance | Properties & Validation | Project Structure

Bonus:

Add a __str__ method for clean object printing. Reading ahead: write a few basic tests using pytest.

Task 5: Weather Dashboard Using a Public REST API

Module 6

Description:

- Consume a public REST API (e.g., OpenWeatherMap) to fetch live weather data for a user-specified city.
- Handle API authentication (API keys) using environment variables — the python-dotenv package is recommended for loading a .env file, but setting variables directly via your OS/terminal is also acceptable.
- Handle error responses (invalid city, timeout, failed request) gracefully, and present the result in a clean, readable CLI format.

Tools & Libraries:

Python | requests | python-dotenv (optional)

Covered Topics

REST APIs | HTTP Methods & Status Codes | JSON Parsing | Secrets Management

Bonus:

- Reading ahead: write automated tests for your API-handling functions using pytest, mocking the API response so tests don't depend on a live network call.

Task 6: Web Scraping and Analysis of Job Postings

Module 7

Description:

- Scrape job listings from a real website (like Indeed or a static HTML page)
- Check the website's robots.txt file to ensure scraping is permitted and to identify which sections of the site are allowed to be scraped.
- Extract fields such as job title, location, company, skills, and posted date
- Clean and analyze the data to find most in-demand roles or skills

Tools & Libraries:

Python | Pandas | BeautifulSoup or Selenium

Covered Topics

Web scraping | Text cleaning | Skill frequency analysis

Bonus:

Visualize most frequent skills by city

Task 7: LLM-Powered Document Q&A Tool (RAG Basics)

Module 8

Description:

- Build a minimal RAG tool using 3–5 short sample text documents (provided or your own).
- Split each document into simple fixed-size chunks, generate embeddings using one embedding method of your choice (e.g., OpenAI's embedding endpoint, or the free local sentence-transformers library), and store them in a basic vector store.
- On a user question, retrieve the most similar chunk(s) via similarity search, pass them to an LLM to generate an answer, and cite which chunk(s) were used.
- Define a clear, configurable similarity-score threshold below which the tool must respond "I don't know" rather than guess — don't rely on prompt wording alone to produce this behavior. Test your tool with at least one question that has no relevant answer in the document set, and confirm it correctly declines to answer.
- Use sample/public documents only — never upload private or sensitive files to a third-party API.

Tools & Libraries:

Python | LLM API or local model | one embedding method | simple vector store (FAISS/Chroma)

Covered Topics

Retrieval-Augmented Generation (RAG) basics | Embeddings & Similarity Search | LLM API Integration | Prompt Engineering | Confidence Thresholds

Bonus:

- Add a simple web interface (Streamlit or Gradio), or extend to a larger/multi-format document set.

Task 8: AI-Assisted Code Review Tool

Module 9

Description:

- Build a script that takes an existing Python file and sends it to an LLM with a structured prompt asking for bug detection, PEP 8 compliance review, and refactor suggestions.
- Parse the LLM's response into a structured report (JSON with issue, line, suggestion fields).
- Validate not just that the response is well-formed JSON, but that each entry actually contains all required fields with valid, non-empty values — discard or flag any entry that fails this check rather than passing it through silently.
- Since LLM-reported line numbers can be inaccurate, allow the tool to mark a line reference as "approximate" when it cannot be verified against the actual file.
- Important: this tool reviews and reports suggestions only — it must not automatically execute or apply any AI-generated code changes.

Tools & Libraries:

Python LLM API

Covered Topics

AI-Assisted Development | Prompt Engineering for Structured Output | Code Quality Review | Output Validation

Bonus:

- Add a feature that shows a before/after diff for a suggestion, which the user can manually choose to accept or reject.

Task 9: AI Agent for a Two-Tool Task

Self-study

Description:

- Build a simple AI agent that takes a natural-language goal and can call exactly two tools you define (e.g., a calculator function and a function that reads from a local file, or a weather lookup and a file-writer).

- Implement a limited execution loop (e.g., max 5 steps) so the agent cannot run indefinitely.

- Log every tool call the agent makes and its result (inputs, outputs, which tool was used) — log the agent's actions and tool calls only, not its internal reasoning or hidden chain-of-thought.

Example workflow (for reference):

Goal: "What's 15% of 240, and save the answer to a file called result.txt?" → Agent calls the calculator tool with 240 * 0.15 → gets 36 → calls the file-writer tool to save 36 to result.txt → logs both tool calls with their inputs/outputs → reports back: "Calculated 36 and saved it to result.txt." Your submission should produce a similarly complete, logged run for your own chosen goal and two tools.

Tools & Libraries:

Python | LLM API | logging

Covered Topics

Basic AI Agents | Tool Use | Execution Logging

Bonus:

- Add a third tool (e.g., a simple web search), and track how many LLM calls were made to complete the task.